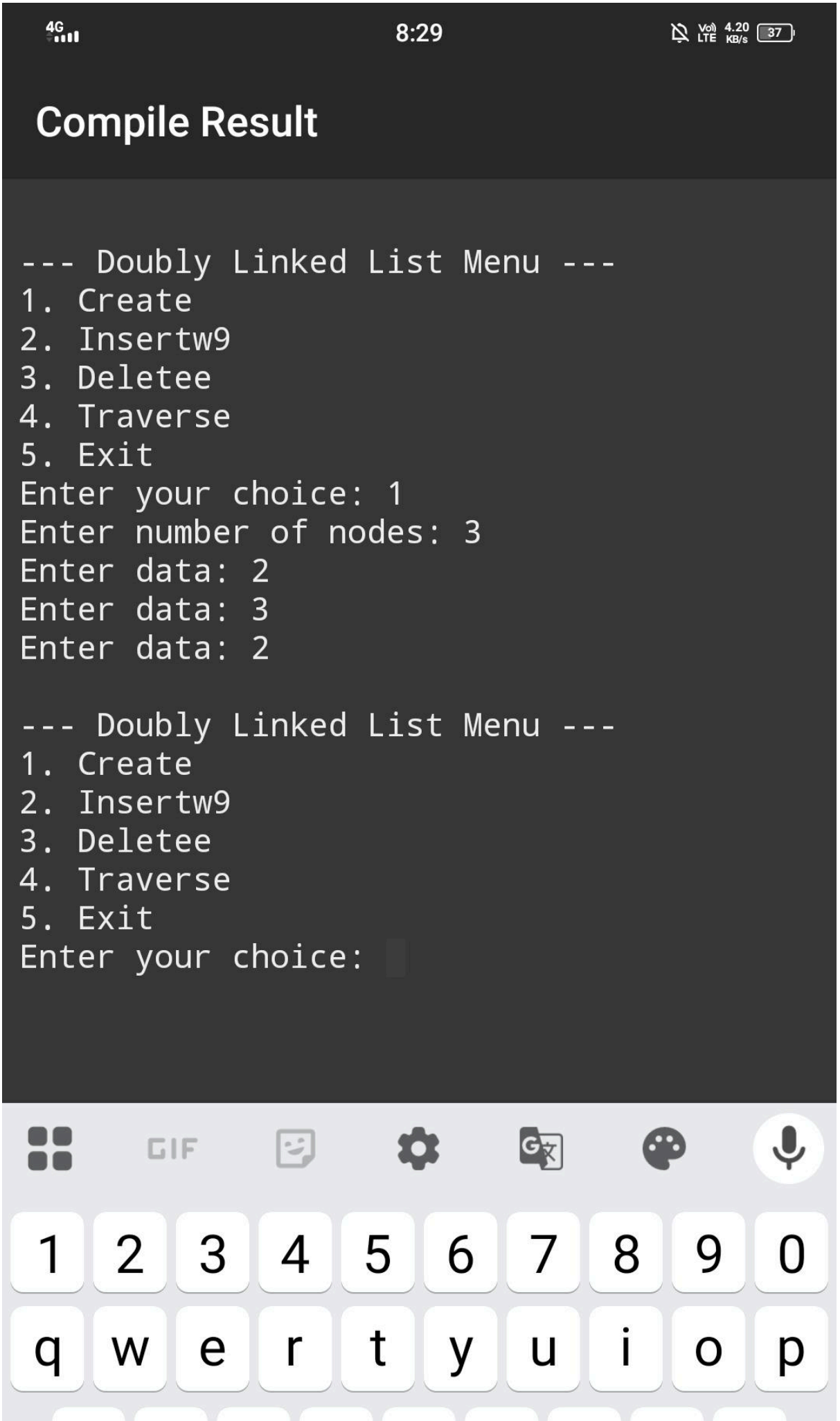
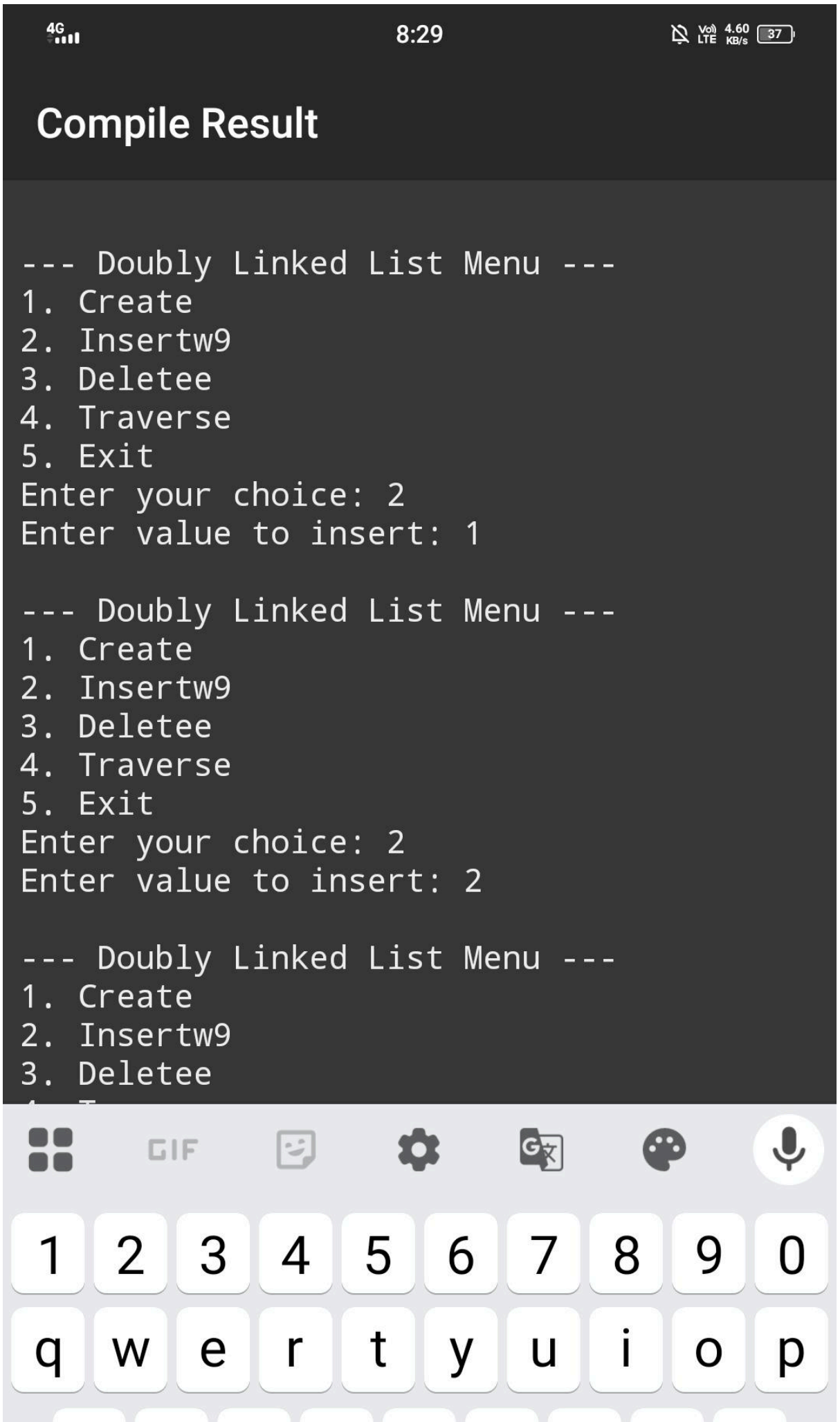
```c
  #include <stdio.h>
#include <stdlib.h>

/* Definition of node */
struct node {
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

/* Function Prototypes */
void create();
void insertw9();
void deletee();
void traverse();

/* Main Function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Doubly Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insertw9\n");
        printf("3. Deletee\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create();
                    break;
            case 2: insertw9();
                    break;
            case 3: deletee();
                    break;
            case 4: traverse();
```

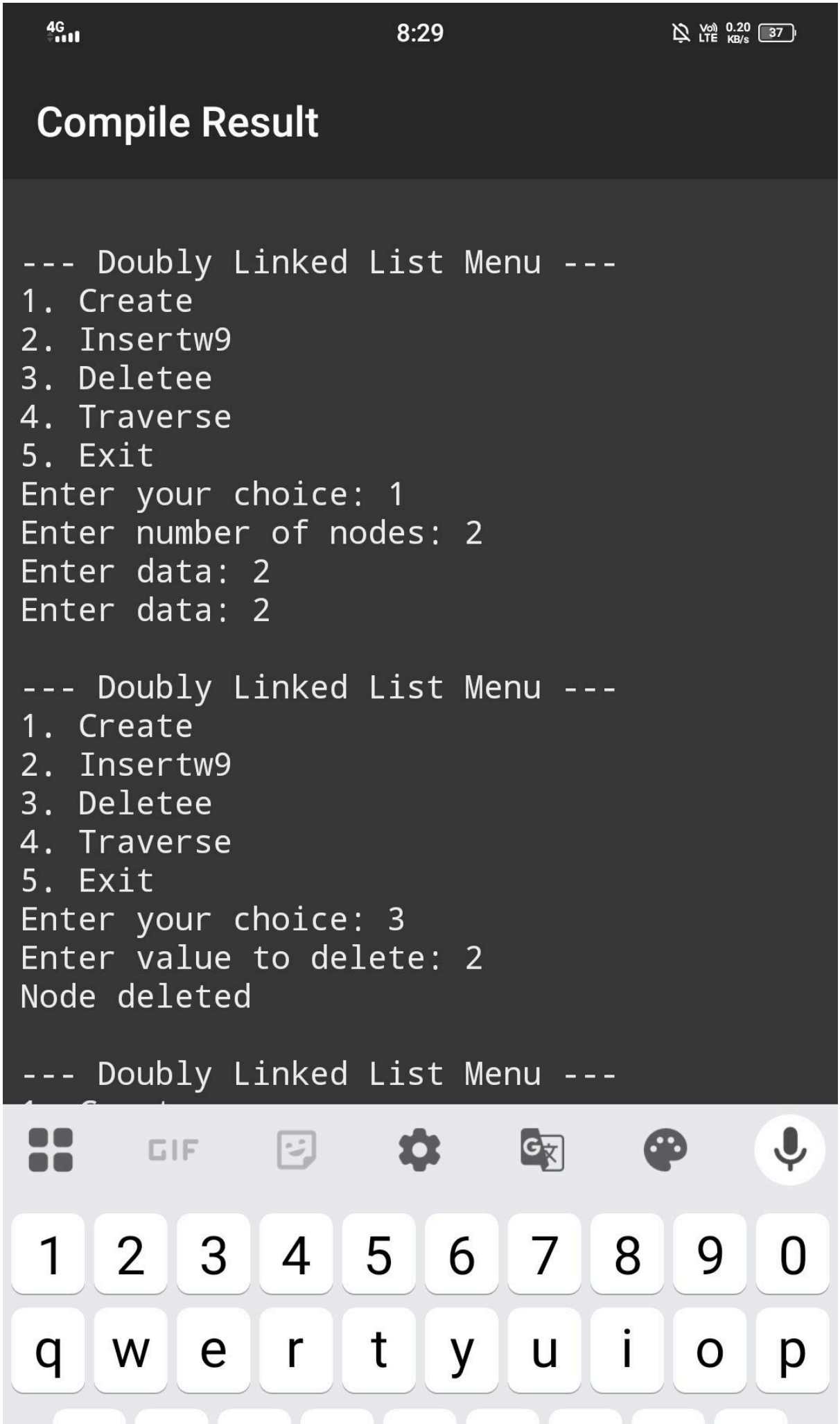
```
                break;
            case 5: exit(0);
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}

/* Create Doubly Linked List */
void create() {
    int n, i, value;
    struct node *temp, *newnode;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->prev = NULL;
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
            newnode->prev = temp;
        }
    }
}

/* Insert at End */
void insertw9() {
    int value;
    struct node *newnode, *temp;

    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &value);

    newnode->data = value;
```

```
    newnode->prev = NULL;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
        newnode->prev = temp;
    }
}

/* Delete a Node by Value */
void deletee() {
    int value;
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &value);

    temp = head;

    /* If first node is to be deleted */
    if (temp->data == value) {
        head = temp->next;
        if (head != NULL)
            head->prev = NULL;
        free(temp);
        printf("Node deleted\n");
        return;
    }

    while (temp != NULL && temp->data != value)
        temp = temp->next;

    if (temp == NULL) {
        printf("Value not found\n");
    } else {
        temp->prev->next = temp->next;
        if (temp->next != NULL)
```

```
            temp->next->prev = temp->prev;
        free(temp);
        printf("Node deleted\n");
    }
}

/* Traverse Forward and Backward */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Forward Traversal: ");
    temp = head;
    while (temp->next != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("%d\n", temp->data);

    printf("Backward Traversal: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}
```


8:29
Compile Result
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 2
Enter data: 3
Enter data: 2
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice:
GIF
1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p


8:29
Compile Result
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 1
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 2
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
GIF
1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p


8:29
Compile Result
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 2
Enter data: 2
Enter data: 2
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 3
Enter value to delete: 2
Node deleted
--- Doubly Linked List Menu ---
GIF
1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p

Compile Result

```
--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 2
Enter data: 2
Enter data: 3

--- Doubly Linked List Menu ---
1. Create
2. Insertw9
3. Deletee
4. Traverse
5. Exit
Enter your choice: 4
Forward Traversal: 2 <-> 3
Backward Traversal: 3 <-> 2 <-> NULL

--- Doubly Linked List Menu ---
```